

Vector stores and retrieval

LESSON 12 · MODULE 3 — KNOWLEDGE & MEMORY

What you'll learn

- The two control points that decide what your agent retrieves: the **vector store** (set at KB creation) and the **retrieval strategy** (set at query time)
- How to use `kb.query()` as a raw retrieval API to debug exactly what the agent sees — chunks plus scores, no agent in the loop
- The four retrieval strategies (`basic`, `mmr`, `hyde`, `time_aware`) and when each one wins
- How to tune retrieval **per call** with `with_config()` without mutating the knowledge base

Key concepts

Stay on the default vector store until you have a reason not to. A knowledge base defaults to **Qdrant** under the hood; the SDK also supports Weaviate, pgvector, Milvus, and Neptune, set with `vector_store=` at creation. The picking rule is short — keep Qdrant unless your team already runs a different vector DB in production. Don't introduce new infrastructure you don't need.

TAKEAWAY

Stay on the default vector store until you have a reason not to.

`kb.query()` is your **retrieval debugger**. When an agent gives a weird answer, the first question is "what did it actually retrieve?" `kb.query()` answers that directly: pass a question, get back ranked chunks with similarity scores and no agent in the loop. The score column is the signal — if your top result is below ~0.3–0.4, retrieval is uncertain and the agent's answer probably is too.

Four retrieval strategies, picked by the shape of your queries.

`basic` is straight vector similarity — fast, predictable, the right default. `mmr` (maximal marginal relevance) adds a diversity penalty so you don't get five near-identical chunks. `hyde` (hypothetical document embeddings) has the model imagine an ideal answer, embed *that*, and search against it — better recall when questions are vague or phrased very differently from the source. `time_aware` weights recency on top of similarity, tunable via `time_decay_factor`, for version docs where you want the latest.

TAKEAWAY

Four retrieval strategies, picked by the shape of your queries.

The picking rule. Default to `basic`. Switch to `mmr` if you're getting redundant chunks, `hyde` if queries are vague and recall is weak, `time_aware` when recency matters. On a small KB the differences are subtle; on a real corpus with near-duplicate chunks, MMR is the difference between five copies of one fact and five different angles.

TAKEAWAY

The picking rule.

Tune retrieval per call, not per knowledge base. `kb.with_config()` returns a wrapped knowledge base with per-call settings that pass through to `agent.run()` and **mutate nothing**. Same KB, three knobs tuned for one specific call — pull more chunks, use MMR for diversity,

TAKEAWAY

Tune retrieval per call, not per knowledge base.

drop low-confidence matches with a score threshold — while the KB itself stays untouched for the next call that wants different settings. That's the level of control you want past the prototype stage.

Code shown in this lesson

PYTHON

```
# Vector store is chosen at KB creation. Default is Qdrant — keep it unless you
# already run another vector DB in production.
kb = studio.create_knowledge_base(name="veristack-docs")
# kb = studio.create_knowledge_base(name="veristack-docs", vector_store="weaviate")
# also: "pgvector", "milvus", "neptune"

# Seed several short docs (SSO, rate limits, data export, audit logs, RBAC) so
# retrieval actually has to choose between topics.
kb.add_text("VeriStack data export: admins can export workspace data from ...")
```

PYTHON

```
# kb.query — raw retrieval, no agent. Returns ranked chunks + scores for debugging.
results = kb.query(
    "How do I export my data?",
    top_k=3,
    retrieval_type="basic",    # default
    score_threshold=0.0,      # default
)
for chunk in results:
    print(round(chunk.score, 2), chunk.text[:60])
# 0.47  data export ...    ← the doc we asked about
# 0.31  audit logs ...    ← plausibly related
# 0.12  sso ...           ← low confidence, basically noise
```

PYTHON

```
# Switch strategy at query time.
results = kb.query("How do exports work?", top_k=3, retrieval_type="mmr")
# Recency-weighted variant:
# kb.query("...", top_k=3, retrieval_type="time_aware", time_decay_factor=0.3)
```

PYTHON

```
# Tune retrieval per call without mutating the KB.
tuned = kb.with_config(top_k=5, retrieval_type="mmr", score_threshold=0.3)
response = agent.run("What roles can I assign?", knowledge_bases=[tuned])
print(response.response) # answer sourced from the RBAC chunk, MMR-tuned for this call
```

TRY THIS

- 1 Build a KB with five short docs on different topics. Run the *same* query through `kb.query()` with `retrieval_type="basic"` and then `"mmr"`. Compare the third result — watch MMR trade a little similarity for diversity.

- 2 Find a query where your top score sits below 0.3. Now rephrase the question to match the source wording and watch the score climb — then try `hyde` on the *original* vague phrasing and see if recall recovers without the rewrite.
- 3 Wrap the KB with `with_config(score_threshold=0.4)` and ask something your corpus doesn't cover. Confirm the agent gets *nothing* back rather than low-confidence noise.